

TEMA 1 – Diseño del Interfaz

Objetivo del tema

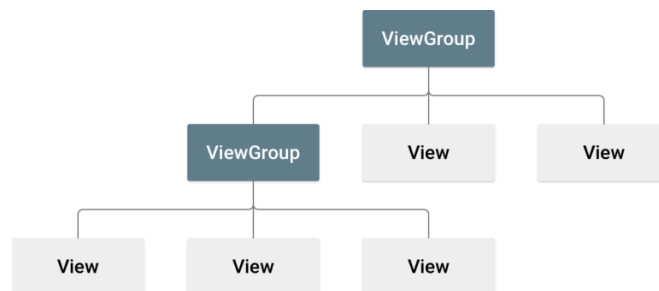
En este tema aprenderás las bases sobre el interfaz de las Apps. Veremos cuáles son sus componentes, cómo se organizan en layouts, y cómo se asignan eventos para añadirles comportamientos en una actividad.

1.1 Vistas, contenedores, layouts

En los ejemplos hechos hasta ahora, hemos visto que las Apps de Android separan las 'piezas' que forman el programa en dos partes: el código fuente y los recursos. En este apartado vamos a centrarnos en la interfaz.

Debes tener en cuenta que todos los elementos que conforman la interfaz visual de usuario de una App son en el fondo clases. Esto significa que se compilan y se ejecutan como cualquier otra clase. Obviamente, esto es transparente para nosotros, que nos limitamos a usar el IDE y simplemente colocamos botones en un layout y le damos a run.

En Android, la interfaz de usuario (UI) es una jerarquía de *diseños* y *widgets*.



- Los *widgets* o vistas son subclases de **View**. Son los componentes de la UI del tipo botones, etiquetas, tablas y cuadros de texto. Ya has utilizado dos Widget en las prácticas anteriores: el **TextView** y el **Button**. Todos ellos tienen sus eventos, sus propiedades, etc. Si ya has creado interfaces gráficas en Java, todo esto te sonará.

Si esto fuera una página HTML normal, te diría que los *widgets* son las etiquetas `<button>`, `<label>`, `<input>`, etc.

- Los *contenedores* son subclases de **ViewGroup**. Su objetivo es contener otros widgets y por supuesto, a otros contenedores. En general se usan para posicionar los distintos elementos en la pantalla.

Si esto fuera una página HTML normal, te diría que los *contenedores* son las etiquetas `<body>`, `<section>`, `<div>`, etc.

- Un layout es en realidad un contenedor. Su propósito es organizarlo todo en una única pantalla visible para el usuario.

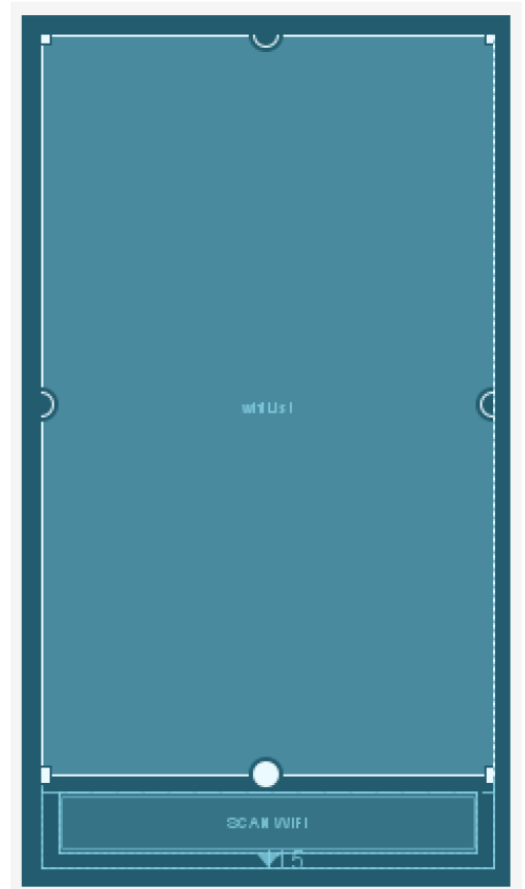
Si esto fuera una página HTML, te diría que los layout son la etiqueta `<HTML>`

- Una **actividad** por tanto estará formada por una pantalla, que generalmente será un layout con todos sus *contenedores* y *widgets*. Es posible que una actividad trabaje con más de un *layout*, pero eso lo veremos más adelante.

Por ejemplo, xml define un interfaz formado por un listado (ViewGroup) y un botón (View) contenido en un RelativeLayout (ViewGroup).

```
<?xml version = "1.0" encoding = "utf-8"?>
<RelativeLayout xmlns:android = "http://schemas.android.com/apk/res/android"
    xmlns:tools = "http://schemas.android.com/tools"
    android:layout_width = "match_parent"
    android:layout_height = "match_parent"
    android:layout_margin = "16dp"
    android:orientation = "vertical"
    tools:context = ".MainActivity">
    <ListView
        android:id = "@+id/wifiList"
        android:layout_width = "match_parent"
        android:layout_height = "match_parent"
        android:layout_above = "@+id/scanBtn" />

    <Button
        android:id="@+id/scanBtn"
        android:layout_width="match_parent"
        android:layout_height="50dp"
        android:layout_alignParentBottom="true"
        android:layout_gravity="bottom"
        android:layout_marginStart="15dp"
        android:layout_marginTop="15dp"
        android:layout_marginEnd="15dp"
        android:layout_marginBottom="15dp"
        android:text="Scan WiFi" />
</RelativeLayout>
```



Todos los componentes visuales en Android son subclases de View. Esto define básicamente un espacio rectangular en pantalla sobre el que podemos pintar cosas e interaccionar con el usuario. Cuando creamos un interfaz, generamos un fichero xml en el que definimos su layout, y después vamos añadiendo los diferentes widgets o vistas que queremos utilizar. Una vez completada la interfaz, pasamos a definir la **activity** a la que está vinculada.

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
```

La segunda línea del onCreate () se emplean para vincular la en layout a la activity. El **setContentView ()** sirve precisamente para eso. Es posible incluso cambiar el layout de una activity dinámicamente en tiempo de ejecución, por ejemplo, al rotar el móvil.

Para referirnos a los diferentes layouts, widgets y contenedores (en realidad, a todo lo que hay en Resources) se emplea la **clase R**. En el caso que estamos tratando, estamos referenciando a un layout llamado **activity_main.xml**.

```
val textView : TextView = findViewById(R.id.textView)
val button : Button = findViewById(R.id.button)
```

Para poder usar cada uno de los diferentes widgets desde nuestra activity, es necesario referenciarlos. Para eso se emplea el método **findViewById ()**. En este caso, la referencia se hace mediante la id del widget. Por tanto, **siempre** hay que dar una id (identificador) a cada uno de los widgets de nuestra interfaz, sin repetir el nombre.

El problema que tiene **findViewById ()** es que siempre retorna una View, por lo que será necesario especificar el tipo de objeto que estamos esperando. Esto sucede tanto en Java como en Kotlin.

Finalmente, la clase R también permite acceder a otros recursos como, por ejemplo, al string.xml.

```
val textHola = getString(R.string.text_hello_world)
```

1.2 Las vistas

Android proporciona un conjunto de vistas o *widgets* por defecto. Y sí, es posible crear tus propios widgets, bajarte otros, extenderlos, etc. Los más comunes son:

- **TextView**: Una etiqueta de solo lectura. Permite multilínea, formateo, etc.
- **EditText**: Caja de texto editable. Permite multilínea, texto flotante, etc.
- **ListView**: Un contenedor que es agrupa widgets en forma de lista vertical.
- **Spinner**: Un combo box de toda la vida, desplegable al pulsarlo.
- **Button**: El botón de toda la vida.
- **Checkbox**: Un botón de dos estados representado con un cuadradito.
- **RadioButton**: Como un checkbox, pero es un circulito. Se puede hacer un Radio-Group, un contenedor de RadioButton en el que solamente se puede pulsar uno.
- **SeekBar**: Una barra de desplazamiento que puede tener cualquier valor.

Estos widgets son básicamente iguales a los paquetes AWT y Swing de Java, o incluso a los de HTML, aunque adaptados a Android Studio.

1.3 Los Layouts

Existen diversos tipos de contenedores. Los más relevantes son:

- **AbsoluteLayout:** Básicamente es un layout que tiene un eje de coordenadas (X, Y). Para colocar cada widget en su interior le tienes que decir dónde ha de colocarlo. Es el menos flexible de todos.
- **Linear Layout:** Es un modelo de cajas, te permite alinear los Widgets en filas horizontales y/o verticales en una columna. Se puede ordenar una fila manipulando el atributo `Gravity`.
- **RelativeLayout:** Define la posición de cada vista como *relativa* a otros elementos equivalentes o en posiciones relativas al **RelativeLayout** padre. Por ejemplo, puedes decir que un **Botón A** está a la izquierda de un **Botón B** y alineado al centro de su **RelativeLayout**.
- **ConstraintLayout:** Es una mejora del **RelativeLayout**, y permite una edición Visual. Define la posición de cada vista a partir de restricciones sobre el diseño principal y las vistas secundarias. Puedes colocar un **Botón A** y un **Botón B** dentro de un **ConstraintLayout** diciendo las distancias que hay entre ellos. Por ejemplo: Botón A está a 16 dp de la parte superior del **ConstraintLayout**, y a 16 dp de su parte izquierda. El Botón B se muestra a 16 dp a la derecha del Botón A, y se muestra alineada con la parte superior del Botón A.
- **ScrollView:** Si bien no es un layout per se, el **ScrollView** nos permite añadir un scroll a un layout para cuando éste resulta ser más grande que la pantalla del móvil. Usarlo es sencillo: añade el **ScrollView**, y anida dentro suyo el layout que quieras usar. Sólo admite uno.

1.4 Los Eventos

En Android, cuando realizamos cualquier clase de acción sobre un *widget*, se ‘dispara’ un **Evento**. Estos **Eventos** tienen un método asociado que se ejecutará cuando se produzca ese **Evento**. Hasta ahora, estábamos relacionando el Evento **onClick ()** de un **Button** con un método que cambiaba el texto de un **EditText**, pero en realidad existen muchos más.

No vamos a repasar la totalidad de los Eventos de cada Widget porque nos podríamos pasar la vida aquí. En clase trabajaremos los más habituales, pero si quieres hacer algo *especialmente raro* te va a tocar buscar en Internet la forma de implementarlo.

Capturar y tratar un evento en Kotlin es tan sencillo como hacer esto:

```
findViewById<Button>( R.id.button ).setOnClickListener {  
  
    // Código de lo que hace el botón  
  
}
```

La arquitectura de Android quiere que en las Actividades especifiques los eventos de los diferentes widgets, pero cualquier lógica de negocio NO debería estar ahí. No obstante, esto no debería preocuparte por ahora.

Ejercicio 5

Crea un pequeño menú para registrar tus gustos en videojuegos. Imita la interfaz que ves en el dibujo. Hay un **RadioGroup**, cuatro **RadioButton** y abajo un **Checkbox**. Según qué opción se escoja, el mensaje cambiará. Por defecto, debe aparecer un mensaje del tipo “A ver cuál escoges”.

El **RadioGroup** debe englobar a los **RadioButton** para evitar que puedan seleccionarse dos de ellos a la vez. Cada vez que se escoja uno, el mensaje cambiará a “Veo que te gusta el...”.

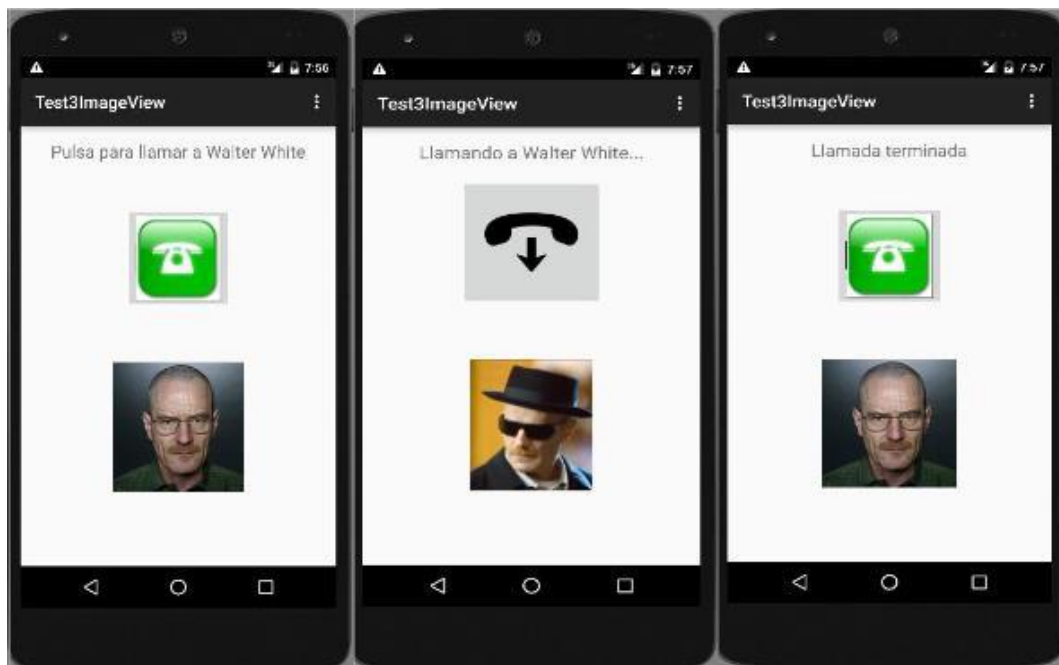


Si se pulsa el Checkbox, el mensaje cambiará a “¡Sí que te mola!”, pero cambiará a “¿No te molan!?” si se le da otra vez.

Esta práctica la tienes solucionada: **P4 - The BestestVideogameKotlin.zip**

Ejercicio 6

Simula una llamada telefónica a Walter White. Crea un interfaz con un **TextView**, un **ImageButton** y un **ImageView**. Fíjate en las siguientes imágenes para ver cómo tienen que cambiar cada uno de los Widgets al pulsar el botón. Investiga por tu cuenta forma de incluir imágenes en el IDE.



Ejercicio 7

Crea una App que sea un login. El usuario deberá introducir el user & pass. Si coincide con el par “admin/admin” muestra un mensaje de bienvenida. Si no, muestra un mensaje de error. Investiga por tu cuenta lo que es un Toast y utilízalo para completar el programa.

Ejercicio 8

Crea una App que permita obtener el nombre, apellido, DNI, sexo y fecha de nacimiento del usuario. El usuario deberá introducir los valores (usa widgets variados). Al pulsar el botón aceptar, se comprueba que:

- El nombre no sea inferior a 2 caracteres, mayor que 15 ni contenga números.
- El apellido no sea inferior a 2 caracteres, mayor que 15, no tenga espacios ni números.
- El DNI debe tener 8 números y una letra
- Al seleccionar el sexo, se puede escoger una imagen genérica de un hombre o una mujer.
- Si es menor de 20 años se muestra un mensaje de error.